

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")

print("Libraries Imported Successfully")
```

Libraries Imported Successfully

```
# Load dataset

df = pd.read_csv('AB_NYC_2019.csv')

# Display first 5 rows

df.head()
```

	id	name	host_id	host_name	neighbourhood_group	neighbourhood
0	2539	Clean & quiet apt home by the park	2787	John	Brooklyn	Kensington
1	2595	Skylit Midtown Castle	2845	Jennifer	Manhattan	Midtown
2	3647	THE VILLAGE OF HARLEM....NEW YORK !	4632	Elisabeth	Manhattan	Harlem
3	3831	Cozy Entire Floor of Brownstone	4869	LisaRoxanne	Brooklyn	Clinton Hi
4	5022	Entire Apt: Spacious Studio/Loft by central park	7192	Laura	Manhattan	East Harlem

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

```
# Shape of dataset

print("Dataset Shape:", df.shape)
```

```
# Dataset information
```

```
df.info()
```

```
# Column names
```

```
print(df.columns)
```

```
# Missing values
```

```
print(df.isnull().sum())
```

```
Dataset Shape: (48895, 16)
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 48895 entries, 0 to 48894
```

```
Data columns (total 16 columns):
```

#	Column	Non-Null Count	Dtype
0	id	48895 non-null	int64
1	name	48879 non-null	object
2	host_id	48895 non-null	int64
3	host_name	48874 non-null	object
4	neighbourhood_group	48895 non-null	object
5	neighbourhood	48895 non-null	object
6	latitude	48895 non-null	float64
7	longitude	48895 non-null	float64
8	room_type	48895 non-null	object
9	price	48895 non-null	int64
10	minimum_nights	48895 non-null	int64
11	number_of_reviews	48895 non-null	int64
12	last_review	38843 non-null	object
13	reviews_per_month	38843 non-null	float64
14	calculated_host_listings_count	48895 non-null	int64
15	availability_365	48895 non-null	int64

```
dtypes: float64(3), int64(7), object(6)
```

```
memory usage: 6.0+ MB
```

```
Index(['id', 'name', 'host_id', 'host_name', 'neighbourhood_group',
       'neighbourhood', 'latitude', 'longitude', 'room_type', 'price',
       'minimum_nights', 'number_of_reviews', 'last_review',
       'reviews_per_month', 'calculated_host_listings_count',
       'availability_365'],
      dtype='object')
```

id	0
name	16
host_id	0
host_name	21
neighbourhood_group	0
neighbourhood	0
latitude	0
longitude	0
room_type	0
price	0
minimum_nights	0
number_of_reviews	0
last_review	10052

```
reviews_per_month      10052
calculated_host_listings_count    0
availability_365        0
dtype: int64
```

```
# Check duplicate rows
```

```
print("Duplicate Rows:", df.duplicated().sum())
```

```
Duplicate Rows: 0
```

```
# Remove duplicates
```

```
df.drop_duplicates(inplace=True)
```

```
print("Duplicates Removed")
```

```
Duplicates Removed
```

```
# Missing values before cleaning
```

```
print(df.isnull().sum())
```

```
id                0
name              16
host_id           0
host_name         21
neighbourhood_group    0
neighbourhood      0
latitude          0
longitude          0
room_type         0
price             0
minimum_nights     0
number_of_reviews    0
last_review       10052
reviews_per_month   10052
calculated_host_listings_count    0
availability_365    0
dtype: int64
```

```
# Fill missing values
```

```
df['name'].fillna('Unknown', inplace=True)
```

```
df['host_name'].fillna('Unknown', inplace=True)
```

```
df['reviews_per_month'].fillna(df['reviews_per_month'].median(), inplace=True)
```

```
print("Missing Values Handled")
```

Missing Values Handled

/tmp/ipykernel_1759/289214865.py:3: FutureWarning: A value is trying to be set o
The behavior will change in pandas 3.0. This inplace method will never work beca

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.met

```
df['name'].fillna('Unknown', inplace=True)
```

/tmp/ipykernel_1759/289214865.py:5: FutureWarning: A value is trying to be set o
The behavior will change in pandas 3.0. This inplace method will never work beca

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.met

```
df['host_name'].fillna('Unknown', inplace=True)
```

/tmp/ipykernel_1759/289214865.py:7: FutureWarning: A value is trying to be set o
The behavior will change in pandas 3.0. This inplace method will never work beca

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.met

```
df['reviews_per_month'].fillna(df['reviews_per_month'].median(), inplace=True)
```

```
# Check missing values again
```

```
print(df.isnull().sum())
```

```
id                                0
name                              0
host_id                           0
host_name                         0
neighbourhood_group              0
neighbourhood                    0
latitude                          0
longitude                         0
room_type                        0
price                             0
minimum_nights                   0
number_of_reviews                 0
last_review                      10052
reviews_per_month                 0
calculated_host_listings_count    0
availability_365                  0
dtype: int64
```

```
# Convert neighborhood names to lowercase
```

```
df['neighbourhood_group'] = df['neighbourhood_group'].str.lower()
```

```
df['room_type'] = df['room_type'].str.lower()
```

```
print("Text Standardization Completed")
```

Text Standardization Completed

Statistical summary

print(df.describe())

	id	host_id	latitude	longitude	price \
count	4.889500e+04	4.889500e+04	48895.000000	48895.000000	48895.000000
mean	1.901714e+07	6.762001e+07	40.728949	-73.952170	152.720687
std	1.098311e+07	7.861097e+07	0.054530	0.046157	240.154170
min	2.539000e+03	2.438000e+03	40.499790	-74.244420	0.000000
25%	9.471945e+06	7.822033e+06	40.690100	-73.983070	69.000000
50%	1.967728e+07	3.079382e+07	40.723070	-73.955680	106.000000
75%	2.915218e+07	1.074344e+08	40.763115	-73.936275	175.000000
max	3.648724e+07	2.743213e+08	40.913060	-73.712990	10000.000000

	minimum_nights	number_of_reviews	reviews_per_month \
count	48895.000000	48895.000000	48895.000000
mean	7.029962	23.274466	1.238930
std	20.510550	44.550582	1.520861
min	1.000000	0.000000	0.010000
25%	1.000000	1.000000	0.280000
50%	3.000000	5.000000	0.720000
75%	5.000000	24.000000	1.580000
max	1250.000000	629.000000	58.500000

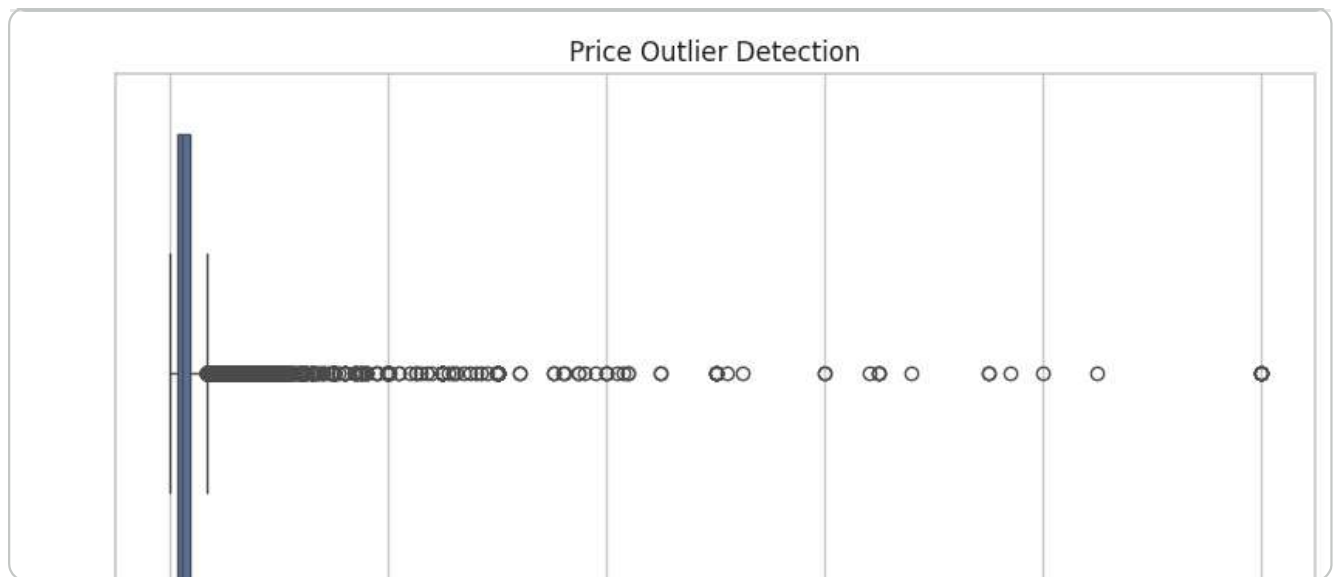
	calculated_host_listings_count	availability_365
count	48895.000000	48895.000000
mean	7.143982	112.781327
std	32.952519	131.622289
min	1.000000	0.000000
25%	1.000000	0.000000
50%	1.000000	45.000000
75%	2.000000	227.000000
max	327.000000	365.000000

```
plt.figure(figsize=(10,5))

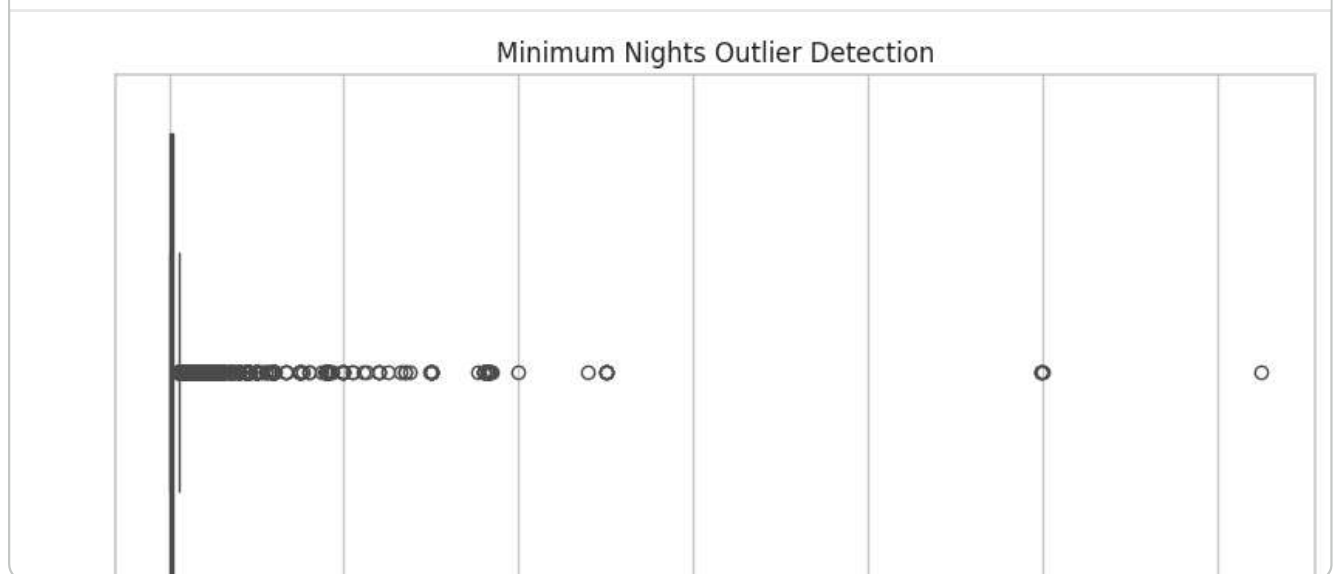
sns.boxplot(x=df['price'])

plt.title('Price Outlier Detection')

plt.show()
```



```
plt.figure(figsize=(10,5))  
  
sns.boxplot(x=df['minimum_nights'])  
  
plt.title('Minimum Nights Outlier Detection')  
  
plt.show()
```



```
# Remove extreme price outliers  
  
df = df[df['price'] < 1000]  
  
# Remove extreme minimum nights outliers  
  
df = df[df['minimum_nights'] < 100]  
  
print("Outliers Removed")
```

Outliers Removed

```
plt.figure(figsize=(8,5))

sns.countplot(x=df['room_type'])

plt.title('Room Type Distribution')

plt.show()
```

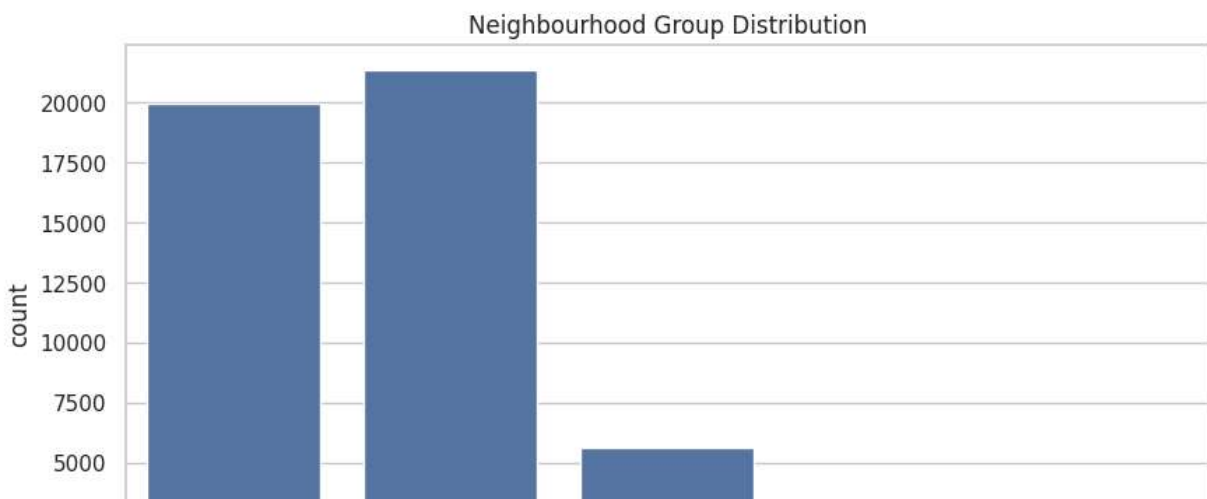


```
plt.figure(figsize=(10,5))

sns.countplot(x=df['neighbourhood_group'])

plt.title('Neighbourhood Group Distribution')

plt.show()
```



```
# Final dataset information

print(df.info())
```

```
# Final shape
```

```
print("Final Dataset Shape:", df.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 48419 entries, 0 to 48894
```

```
Data columns (total 16 columns):
```

#	Column	Non-Null Count	Dtype
0	id	48419 non-null	int64
1	name	48419 non-null	object
2	host_id	48419 non-null	int64
3	host_name	48419 non-null	object
4	neighbourhood_group	48419 non-null	object
5	neighbourhood	48419 non-null	object
6	latitude	48419 non-null	float64
7	longitude	48419 non-null	float64
8	room_type	48419 non-null	object
9	price	48419 non-null	int64
10	minimum_nights	48419 non-null	int64
11	number_of_reviews	48419 non-null	int64
12	last_review	38611 non-null	object
13	reviews_per_month	48419 non-null	float64
14	calculated_host_listings_count	48419 non-null	int64
15	availability_365	48419 non-null	int64

```
dtypes: float64(3), int64(7), object(6)
```

```
memory usage: 6.3+ MB
```

```
None
```

```
Final Dataset Shape: (48419, 16)
```

```
# Save cleaned dataset
```

```
df.to_csv('cleaned_AB_NYC_2019.csv',
          index=False)
```

```
print("Cleaned Dataset Saved Successfully")
```

```
Cleaned Dataset Saved Successfully
```

Business Insights

1. Missing Data

Several columns contained missing values, especially `reviews_per_month`, which were handled using median imputation.

2. Duplicate Records

Duplicate rows were identified and removed to maintain data integrity.

3. Outliers

Extreme values in price and minimum_nights were detected and removed to improve data quality.

4. Standardization

Text formatting inconsistencies were standardized using lowercase conversion.

5. Dataset Quality

The cleaned dataset became more accurate, reliable, and suitable for further analysis and machine learning applications.

Recommendations

1. Regular Data Validation

Implement validation checks during data collection to reduce missing and inconsistent values.

2. Duplicate Prevention

Use unique identifiers to avoid duplicate records in the future.

3. Outlier Monitoring

Monitor unusually high prices and minimum nights values regularly.

4. Standardized Input Formats

Maintain consistent formatting standards for text-based data fields.

5. Continuous Data Cleaning

Perform periodic data cleaning to maintain high-quality datasets for analysis and business decisions.

Conclusion

The Data Cleaning project successfully improved the quality and reliability of the Airbnb NYC dataset by handling missing values, removing duplicate records, standardizing text data, and detecting outliers.

The project demonstrated practical skills in data preprocessing and cleaning using Python libraries such as Pandas, Matplotlib, and Seaborn. The cleaned dataset is now more suitable for accurate analysis, visualization, and machine learning applications.